Curso: Heurística
Profesor: Juan Carlos Rivera Agudelo
Febrero 3 de 2026
Métodos constructivos y aleatorizados

METODOS CONSTRUCTIVOS Y ALEATORIZADOS

Teniendo en cuenta el problema asignado para trabajar durante el semestre (No-Wait Job Shop Scheduling Problem with Release Dates and Flow Time Minimization, ver Anexo 1), se deben implementar tres algoritmos: un algoritmo constructivo y dos algoritmos constructivos aleatorizados (construcción GRASP, construcciones con ruido, construcciones basadas en colonias de hormigas). Los algoritmos deben poderse parametrizar para ejecutarse con diferentes valores de cada parámetro según sea el caso (ver Anexo 4 – ejemplo de definición de parámetros).

El algoritmo debe ser desarrollado en Python (otros lenguajes pueden ser utilizados sujetos a previa aprobación por parte del profesor).

Las pruebas del algoritmo deben ser realizadas con las instancias de prueba disponibles en las librerías para el problema seleccionado. Las instancias están disponibles en Interactiva Virtual con el formato descrito en el Anexo 2.

Cuando los algoritmos desarrollados sean basados en ideas de otros autores, i.e. método de los ahorros, NEH u otros, se debe indicar explícitamente en la descripción del método.

Formato de Entrega:

1. Realizar una presentación, en diapositivas, en la que se incluya:
   - Descripción de los algoritmos implementados.
   - Descripción de los resultados obtenidos, incluyendo:
     - Comparación con una cota inferior.
     - Comparaciones utilizando diferentes valores para cada parámetro.
     - Comparación entre los métodos implementados.
     - Tiempo de cómputo.
   - Conclusiones.
   - Se deben incluir todas las referencias bibliográficas que hayan servido de apoyo.
   - Se pueden incluir otras secciones en el informe (opcional).
2. Enviar por Interactiva Virtual los archivos correspondientes al código, los archivos de resultados y la presentación (con ayudas audiovisuales, formato libre).
3. Se realizará una sustentación oral individual en horario acordado con el profesor.

Se debe adjuntar los archivos correspondientes a los códigos de los algoritmos. Éstos deben poderse ejecutar sin necesidad de ingresar datos o modificaciones adicionales de forma manual. El algoritmo debe producir, para cada instancia disponible, un archivo de resultado con el nombre y formato descrito en el Anexo 3.

Curso: Heurística
Profesor: Juan Carlos Rivera Agudelo
Febrero 3 de 2026
Métodos constructivos y aleatorizados

Evaluación:

18%	Método constructivo
18%	Método aleatorizado 1
18%	Método aleatorizado 2
5%	Cota inferior
10%	Comparación de parámetros
10%	Comparación entre métodos
6%	Comparación de tiempos de cómputo
5%	Conclusiones
10%	Validación de resultados
10%	Ranking*

* 10% adicional para el estudiante con la mejor solución, 5% para el segundo lugar, 2.5% para el tercer lugar.

FECHA DE ENTREGA: La fecha límite de entrega es el miércoles 11 de marzo de 2026 (vía Buzón de Interactiva Virtual).

Curso: Heurística
Profesor: Juan Carlos Rivera Agudelo
Febrero 3 de 2026
Métodos constructivos y aleatorizados

ANEXO 1

DEFINICIÓN Y FORMULACIÓN DEL PROBLEMA

The no-wait job shop scheduling problem (NWJSSP) is a specific case of the job shop scheduling (JSSP) where waiting time is not allowed between any two consecutive operations for a job. The no-wait characteristics mean that once started, a job must process all its operations one by one according to its processing route without any interruption. Such no-wait constraints widely exist in practical production environment such as the steel-making industry (Pinedo [1]; Tang et al. [2]), concrete manufacturing (Grabowski and Pempera [3]; Deng et al. [4]), chemical and pharmaceutical industries (Rajendran [5]), and food industries (Hall and Sriskandarajah [6]).

Problem statement

The no-wait job shop scheduling problem involves a set of machines M and a set of jobs J which have to be processed on these machines. Each job $j \in J$ is characterized by a release date r_j and a set of operations O. Each operation $u \in O$ is also characterized by the machine on which it is processed (m_{ju}), as well as the processing time it requires (p_{ju}).

The following constraints must be met: (1) Each job has its own sequence of operations, which indicates its own processing route; i.e. operation u can only be processed after operation $u - 1$ ($u \in O$, $u > 1$). (2) No operation can be interrupted once started. (3) No job can be processed on more than one machine simultaneously. (4) No machine can process more than one job simultaneously. (5) No job is allowed to reenter previous machines. (6) No waiting time is allowed between two consecutive operations of the same job.

The objective is to find a feasible schedule that minimize the sum of completion time of all jobs, namely, the total flow time.

Curso: Heurística
Profesor: Juan Carlos Rivera Agudelo
Febrero 3 de 2026
Métodos constructivos y aleatorizados

ANEXO 2

FORMATO DE DATOS DE LAS INSTANCIAS

En el archivo "NWJSSP Instances.zip" (disponible en EAFIT Interactiva) se encuentran 15 instancias con datos para el NWJSSP, donde el número de trabajos y máquinas varía entre 10 y 1000.

La primera fila indica el número de trabajos (n) y el número de máquinas (m).

Las siguientes n filas indican, para cada trabajo $i \in J$, la información correspondiente a las operaciones de cada trabajo. Cada fila contiene m pares de valores (uno por cada operación); el primer valor corresponde al índice de la máquina y el segundo corresponde al tiempo de procesamiento. El último valor de cada fila corresponde al tiempo de liberación.

A continuación se presenta un ejemplo con $n = 5$ y $m = 3$:

5	3					
0	1	2	2	1	2	0
1	3	2	3	0	2	0
2	2	1	2	0	3	3
0	3	1	2	2	1	5
0	1	2	1	1	3	0

Curso: Heurística
Profesor: Juan Carlos Rivera Agudelo
Febrero 3 de 2026
Métodos constructivos y aleatorizados

ANEXO 3

FORMATO DE ARCHIVO DE RESULTADOS

Los resultados obtenidos se deben registrar en un archivo de Excel en el cual cada hoja contenga los resultados de cada instancia. El nombre del archivo debe ser NWJSSP_<nombre_estudiante>_<método>.xlsx, donde <nombre_estudiante> debe ser reemplazado por el nombre (nombre, o apellido, o iniciales) de cada estudiante, y <método> debe ser reemplazado por un nombre indicador del método utilizado para hallar los resultados (constructivo, GRASP, ACO, LS, VNS, GA,…). Cada hoja del archivo de Excel debe ser nombrada con el nombre de la instancia respectiva (usar los mismos nombres utilizados en el archivo de datos, sin la extensión txt).

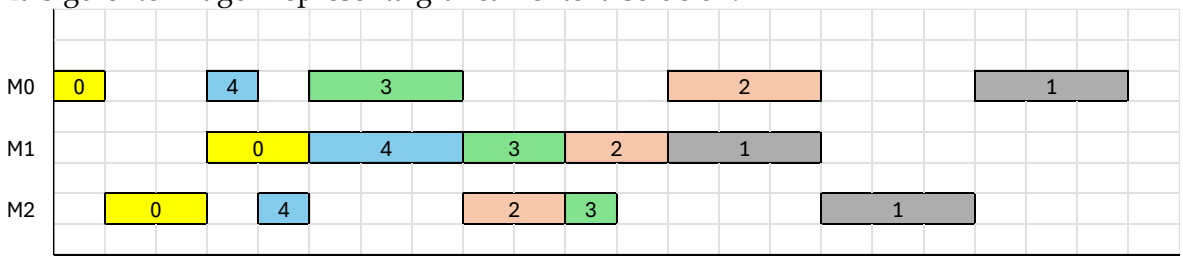
Los resultados deben tener el siguiente formato: La primera fila debe contener 2 valores: el valor de la función objetivo (Z) y tiempo de cómputo (en milisegundos redondeado al entero más cercano). La siguiente fila debe contener el tiempo de inicio de cada trabajo.

El siguiente es un ejemplo de una solución factible para la instancia de ejemplo presentada en el Anexo 2.

60	0			
0	12	8	5	3

En la solución anterior el tiempo de flujo es de 60 unidades, y el tiempo de cómputo es 0 milisegundos. El primer trabajo inicia en el periodo 0, el segundo inicia en el periodo 12, el tercero en el periodo 8, el cuarto en el periodo 5, y el quinto en el periodo 3.

La siguiente imagen representa gráficamente la solución:



Curso: Heurística
Profesor: Juan Carlos Rivera Agudelo
Febrero 3 de 2026
Métodos constructivos y aleatorizados

ANEXO 4

EJEMPLO DE DEFINICIÓN DE PARÁMETROS

La siguiente figura muestra un algoritmo en lenguaje de programación Python donde luego de cargar librerías y funciones se definen los valores de los 4 parámetros a utilizar: nsol, alpha, K y r. En los algoritmos implementados los parámetros deben tener valores constantes; no deben ser leídos de algún archivo, ni solicitados al usuario en cada ejecución.

```python
import xlwt
from xlwt import Workbook

from Constructive import Constructive

from GRASP1 import GRASP1
from GRASP2 import GRASP2
from Noise import Noise


# Data reading
from Read import read_fsspsc


wb = Workbook()
sheet1 = wb.add_sheet('Randomized')

nsol=100    # Number of solutions
alpha=0.05  # GRASP parameter
K=5         # Maximum RCL size
r=5         # Noise

for id in range(1,21):
    print(id)
    n,m,L,dur = read_fsspsc(id)

    Z0,S0,I0,F0,Fi0,t0=Constructive(n,m,dur,L,id)
    print("C:\t",Z0,"\t",t0)

    Z1,S1,I1,F1,Fi1,t1=GRASP1(n,m,dur,L,id,alpha,nsol)
    print("GRASP1:\t",Z1,"\t",t1)

    Z2,S2,I2,F2,Fi2,t2=GRASP2(n,m,dur,L,id,K,nsol)
    print("GRASP2:\t",Z2,"\t",t2)

    Z3,S3,I3,F3,Fi3,t3=Noise(n,m,dur,L,id,r,20)
    print("Noise:\t",Z3,"\t",t3)
```